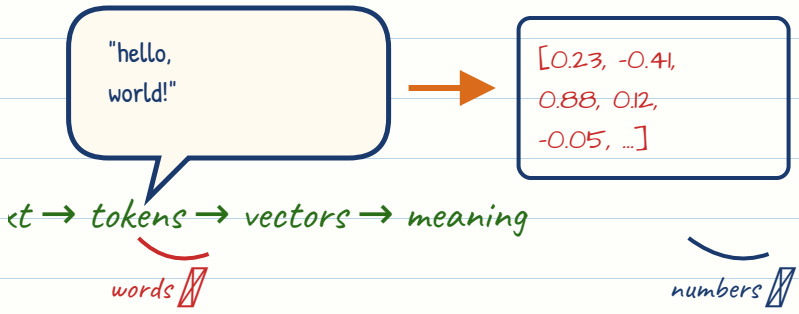
Natural Language Processing

~ handwritten notes ~



contents:

1. Preprocessing · 2. Tokenization · 3. BoW & TF-IDF
4. N-gram LMs · 5. Embeddings · 6. RNN / LSTM
7. Seq2Seq & Attention · 8. Transformers
9. BERT vs GPT · 10. LLMs · 11. NLP Tasks
12. Evaluation · 13. Big Picture

— Sreemanti's notes ✏

1. Preprocessing

Raw text is messy. Clean it before feeding to a model.

✦ Common Steps

- Lowercase → "Apple" = "apple" (lose case info though!)
- Remove punctuation / digits / HTML
- Strip stopwords ("the", "is", "and"...)

  ↑ careful — they matter for some tasks!
- Normalize — Unicode, accents, contractions ("don't" → "do not")

✦ Stemming vs Lemmatization

Stemming

chop suffixes, fast & crude

"running, runs, ran" → "run"

"studies" → "studi" (!)

Lemmatization

use dictionary + POS

"ran" → "run", "better" → "good"

slower, more accurate ✓

✦ Sentence & Word Splitting

Easy in English (split on whitespace + punctuation) ... but tricky cases:

- "U.S.A." vs end of sentence
- "don't" → "do" + "n't"?
- Chinese / Japanese — no spaces between words!
- URLs, emojis ☹, hashtags

★ *Rule:* preprocessing should match what your model was trained on. Pre-trained models often have their own tokenizer — use it!

2. Tokenization

Split text into discrete units (tokens). Vocab size matters!

✦ Three Granularities

- Word-level — "I love NLP" → ["I", "love", "NLP"]

  large vocab, OOV problem for rare words

- Character-level — "love" → ["l","o","v","e"]

  tiny vocab, very long sequences □

- Subword — "unhappiness" → ["un","happi","ness"]

  best of both — modern default ✓

✦ Subword Algorithms

► Byte-Pair Encoding (BPE)

1. start with character vocab
2. find most frequent adjacent pair → merge into one token
3. repeat until vocab size reached

Used by GPT-2, RoBERTa, etc.

BPE merging "low low lower"

step 0: l o w · l o w · l o w e r ↑ "l+o" most frequent

step 1: lo w · lo w · lo w e r ↑ then "lo+w"

step 2: low · low · low e r

...continue until target vocab size

► WordPiece (BERT)

like BPE but merges to maximize likelihood, not frequency. Prefixes subwords with ##:

"playing" → ["play", "##ing"]

► SentencePiece / Unigram

works on raw text (incl. spaces) → language-agnostic. Used by T5, ALBERT, mBART.

> Modern tokenizers can squeeze a long text into a few hundred tokens efficiently & handle any unicode input — no more <UNK> tokens!

3. BoW & TF-IDF

Classic ways to turn a document into a vector — ignore order, count words.

✦ Bag of Words (BoW)

Vocab = all unique tokens. Each doc → vector of token counts.

d1: "cat sat on mat"

d2: "dog sat on rug"

	cat	dog	sat	on	mat
d1	1	0	1	1	1
d2	0	1	1	1	0

✦ TF-IDF

Common words like "the" appear everywhere → not informative. Down-weight them!

$$tf(t, d) = count(t \text{ in } d) / |d|$$

$$idf(t) = \log(N / df(t)) \quad \leftarrow \text{rare = bigger}$$

$$tfidf(t, d) = tf(t, d) \cdot idf(t)$$

Each doc becomes a sparse vector $\in \mathbb{R}^{|V|}$. Compare with cosine similarity:

$$\cos(a, b) = (a \cdot b) / (\|a\| \cdot \|b\|)$$

✓ Pros

- *simple, fast, interpretable*
- *strong baseline for classification & retrieval*
- *still used in search engines!*

✗ Cons

- *ignores word order & syntax*
- *no semantic similarity (cat ≠ kitten)*
- *huge sparse vectors*

4. N-gram Language Models

A language model assigns probabilities to sequences:

$$P(w_1, w_2, \ldots, w_T) = \prod_t P(w_t \mid w_{1:t-1})$$

✦ Markov Assumption

Approximate with only previous (n−1) words:

$$P(w_t \mid w_{1:t-1}) \approx P(w_t \mid w_{t-n+1:t-1})$$

- *unigram*: $P(w)$ — just word frequency
- *bigram*: $P(w_t \mid w_{t-1})$
- *trigram*: $P(w_t \mid w_{t-2}, w_{t-1})$

Estimate from counts:

$$P(w_t|w_{t-1}) = \text{count}(w_{t-1}, w_t) / \text{count}(w_{t-1})$$

✦ Smoothing — the Zero Problem

Many real n-grams have count 0 → probability 0 → log = −∞! Fix it:

- *Add-1 (Laplace)*: add 1 to every count

  $$P = (c(x) + 1) / (N + V)$$

- *Add-k*: smaller pseudo-count
- *Back-off*: if trigram missing, fall back to bigram
- *Interpolation*: λ_1 unigram + λ_2 bigram + λ_3 trigram
- *Kneser-Ney* — gold standard, discounts & redistributes

✦ Perplexity

Standard LM evaluation metric. Lower = better.

$$PPL(W) = P(w_1 \ldots w_N)^{-1/N} = 2^{-(1/N)\,\Sigma \log_2 P(w_i|\cdot)}$$

→ average branching factor of the model.

★ *Limits:* n-grams can't generalise across similar words ("happy"/"glad"), can't model long-range dependencies, & vocab grows exponentially with n. Neural LMs fix this!

5. Word Embeddings

Idea → represent each word as a dense vector in $\mathbb{R}^d$ ($d \approx 50$-300) so that similar words are near each other.

✦ Distributional Hypothesis

"You shall know a word by the company it keeps." — Firth, 1957

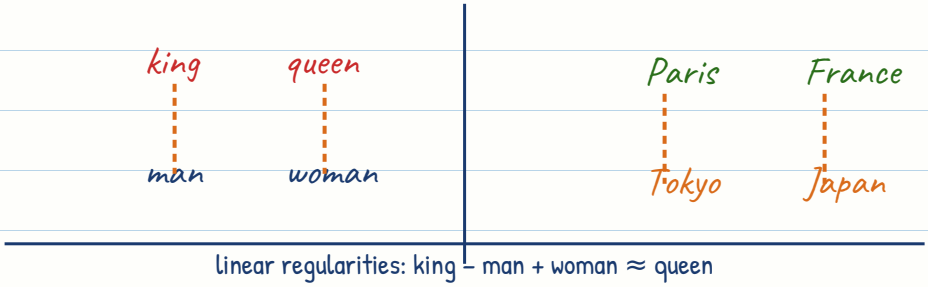
✦ Word2Vec (Mikolov, 2013)

Train a shallow NN to predict context from word (or vice versa).

CBOW
context → centre word
"the ___ sat on" → "cat"

Skip-gram
centre word → context
"cat" → "the", "sat", "on"

Trick: use negative sampling instead of full softmax.



✦ GloVe (Pennington, 2014)

Factorise the global word co-occurrence matrix → captures statistics from whole corpus.

$$J = \sum_{i,j} f(X_{ij}) \left(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij}\right)^2$$

✦ Limits → Contextual Embeddings

Word2Vec / GloVe give one vector per word. But "bank" 🏦 ≠ "bank" 🌊

→ ELMo, BERT give different vectors per context!

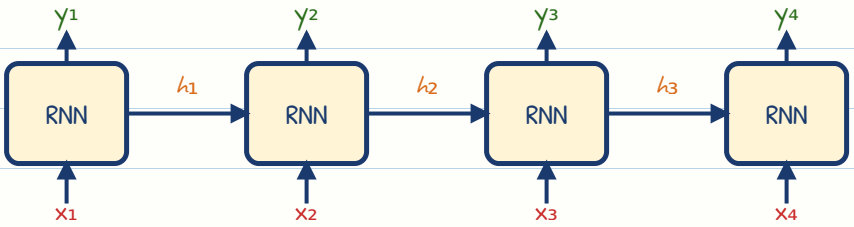
6. RNN, LSTM & GRU

Process sequences by keeping a hidden state that carries info forward.

✦ Vanilla RNN

$$h_t = \tanh(W_h\, h_{t-1} + W_x\, x_t + b)$$

$$y_t = W_y\, h_t + b_y$$



Problem: gradients vanish / explode through time → can't learn long dependencies 😢

✦ LSTM — Long Short-Term Memory

Add a cell state C_t + 3 gates that decide what to remember / forget / output.

$$f_t = \sigma(W_f\, [h_{t-1}, x_t]) \quad \text{forget gate}$$

$$i_t = \sigma(W_i\, [h_{t-1}, x_t]) \quad \text{input gate}$$

$$o_t = \sigma(W_o\, [h_{t-1}, x_t]) \quad \text{output gate}$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tanh(W_c [h_{t-1}, x_t])$$

$$h_t = o_t \odot \tanh(C_t)$$

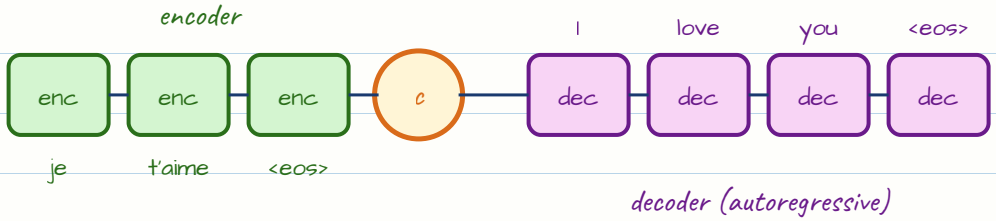
✦ GRU — simpler cousin

2 gates (reset + update). Fewer params, similar perf.

7. Seq2Seq + Attention

✦ Encoder-Decoder

Map a source sequence to a target sequence of different length. Used for translation, summarization, dialogue.



Problem: a single context vector c is a bottleneck for long sentences.

✦ Attention to the Rescue!

At each decoder step, look back at all encoder states. Compute weights over them.

$e_{tj} = \text{score}(s_{t-1}, h_j)$ (similarity)

$\alpha_{tj} = \text{softmax}_j(e_{tj})$ (weights)

$c_t = \sum_j \alpha_{tj} h_j$ (weighted ctx)

Score functions:

- additive (Bahdanau): $v^T \tanh(W[s, h])$
- dot product (Luong): $s^T h$
- scaled dot product: $(s^T h) / \sqrt{d}$ ← used in Transformer!

Side benefit: attention weights are interpretable → see which source words the decoder "looks at"

8. Transformers

"Attention is all you need" — Vaswani et al., 2017

Throw away recurrence. Use only attention (+ feedforward layers). Parallel & scalable!

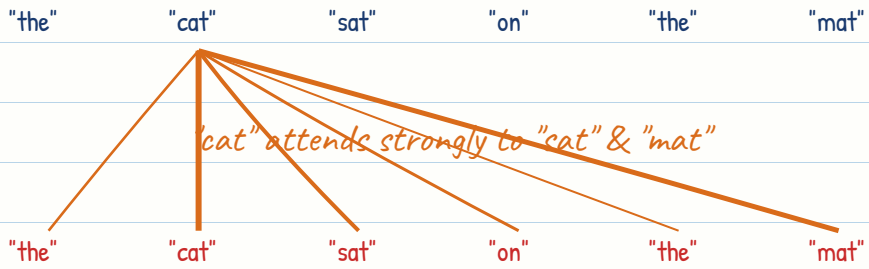
✦ Self-Attention

Each token attends to every other token in the sequence (including itself).

Project each input into Query / Key / Value vectors:

$$Q = X W^Q, \quad K = X W^K, \quad V = X W^V$$

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) V$$



✦ Multi-Head Attention

Run h parallel attention "heads" with different projections → concatenate → linear.

$$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h) W^O$$

Different heads capture different relationships (syntax, coreference, ...).

✦ Positional Encoding

Attention is permutation-invariant → must add position info!

$$PE(pos, 2i) = \sin(pos / 10000^{2i/d})$$

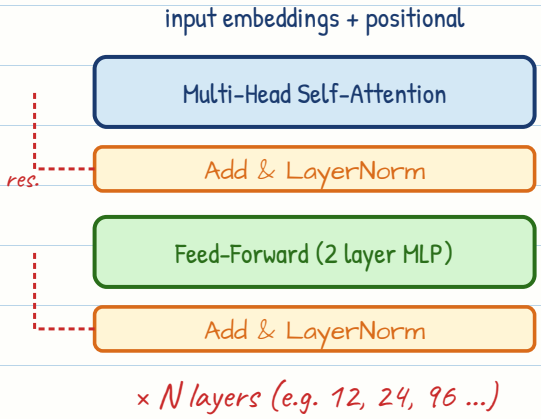
$$PE(pos, 2i+1) = \cos(pos / 10000^{2i/d})$$

Modern variants: learned, rotary (RoPE), ALiBi.

9. Architecture & Variants

One Transformer Block

The "Lego" piece used everywhere:



Three Flavours

Encoder-only
bidirectional attention
→ understand text
BERT, RoBERTa

Decoder-only
causal (masked) attention
→ generate text
GPT, LLaMA, Claude

Encoder-Decoder: both — input is encoded bidirectionally, output is decoded causally.
TS, BART, mTS — great for translation & summarisation.

Why Transformers Won

- parallel — train on GPUs efficiently (RNN must wait for h_{t-1})
- long-range deps in O(1) layer (RNN is O(N))
- scales beautifully — more params + data = better
- same architecture works on text, code, images, audio, proteins...

10. BERT vs GPT

✦ BERT (2018, Devlin et al.)

Bidirectional Encoder Representations from Transformers

Pre-training objectives:

- Masked Language Model (MLM) — hide 15% of tokens, predict them

  "the cat [MASK] on the mat" → "sat"

- Next Sentence Prediction (NSP) — given A, B; is B the next sentence?

  (later models dropped this)

Use it for: classification, NER, QA — anything where you need to understand text.

✦ GPT (2018-, OpenAI)

Generative Pre-trained Transformer

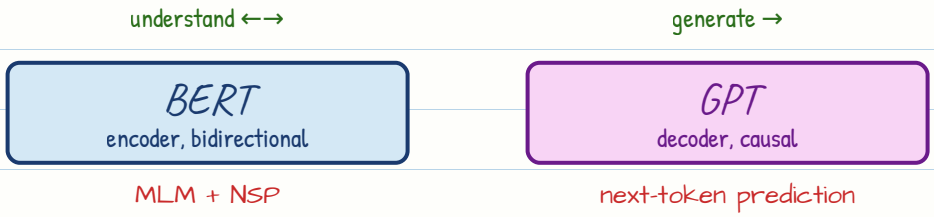
Pre-training objective = next-token prediction:

$$L = -\sum_t \log P(w_t \mid w_1, \ldots, w_{t-1})$$

Causal mask → can only see past. Decoder-only.

Use it for: generation, chat, code, reasoning, ... pretty much everything now.

✦ Side-by-Side



✦ Fine-tuning Recipe

1. start with pretrained checkpoint
2. add a small task head (classifier / token-tagger / etc.)
3. train on your task data with small LR (≈ 2e-5)
4. regularise — frozen layers, dropout, early stop

Modern alternative: LoRA adapters — train tiny low-rank matrices, freeze the rest.

11. Modern LLMs

Scale up GPT-style models & magic happens. Emergent abilities → few-shot learning, reasoning, instruction following.

The 3-Stage Recipe



trillion-token corpus → instruction-tuned → helpful & safe

Pre-training

Self-supervised next-token prediction on a giant corpus (web, books, code).

Cost: months of GPU/TPU time, $$$. Power-law scaling laws:

$$\text{Loss} \propto C^{-\alpha} \quad (C = \text{compute})$$

Instruction Tuning (SFT)

Fine-tune on (instruction → ideal response) pairs collected from humans.

RLHF / DPO

Align outputs with human preferences:

1. collect pairs (response A, response B); humans pick winner
2. train reward model $R(x, y)$
3. optimise LLM with PPO to maximise R (or use DPO closed-form)

Prompting Tricks

- *zero-shot* — just ask
- *few-shot* — include examples in prompt
- *chain-of-thought* — "let's think step by step" → reasoning ↑
- *RAG* — retrieve docs, stuff into context → factual
- *Tool use / function calling* — model decides to call APIs

★ *Watch out:* hallucinations, prompt injection, stale knowledge, bias, jailbreaks. Always verify factual claims! □

12. Classic NLP Tasks

Text Classification

Map document → label (spam, sentiment, topic).

Models: logistic on TF-IDF → fine-tuned BERT → LLM zero-shot.

Named Entity Recognition (NER)

Tag each token with an entity type (or O = none).

Apple	was	founded	by	Steve	Jobs	in	Cupertino
B-ORG	O	O	O	B-PER	I-PER	O	B-LOC

BIO/BIOES tagging. CRF on top of biLSTM/BERT is classic.

Machine Translation

Source language → target language. Seq2seq with attention → Transformer encoder-decoder. Modern: M2M-100, NLLB (200 langs!).

Question Answering

- Extractive — find answer span in passage (SQuAD)
- Generative — synthesize answer (LLM)
- Open-domain — also retrieve relevant docs (RAG)

Summarization

- Extractive — pick salient sentences from input
- Abstractive — generate new text (BART, PEGASUS, LLMs)

Other Goodies

- POS tagging — noun/verb/adj per token
- Dependency parsing — grammatical structure
- Coreference — "he" → "John"
- Topic modelling — LDA, BERTopic
- Text generation, paraphrasing, style transfer

13. Evaluation

Classification Metrics

(same as ML) accuracy, precision, recall, F1, AUC.

Watch for class imbalance — macro-F1 often safer than accuracy.

Language Modelling

- Perplexity — exp(avg cross-entropy). Lower is better.
- Bits-per-character — for char models

Machine Translation: BLEU

Measure n-gram overlap with reference translations:

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_n w_n \log p_n \right)$$

BP = brevity penalty (don't reward too-short outputs).

↑ correlates with human judgement, but only weakly. Newer: chrF, COMET.

Summarization: ROUGE

- ROUGE-N — n-gram recall vs reference
- ROUGE-L — longest common subsequence
- ROUGE-S — skip-bigram

Embedding-based

- BERTScore — cosine sim of BERT embeddings
- BLEURT — fine-tuned regression model
- METEOR — uses synonyms + stemming

LLM Evaluation

- benchmarks: MMLU, BIG-bench, HellaSwag, GSM8K (math), HumanEval (code)
- LLM-as-judge — use a strong model to rate outputs ⚖
- Human eval — still the gold standard 🧑‍⚖️
- Red teaming — find harmful / wrong outputs

★ Trap: automatic metrics ≠ user satisfaction. A high-BLEU translation can still sound robotic. Always look at outputs!

Big Picture

The Pipeline



History (rough!)

- 1950s-90s — rules & grammars
- 1990s-2000s — statistical (HMM, CRF, n-gram LMs)
- 2013 — Word2Vec → dense embeddings
- 2014-17 — RNN, LSTM, seq2seq + attention
- 2017 — Transformer (Attention is all you need)
- 2018 — BERT & GPT-1 → pretrain-then-finetune era
- 2020+ — GPT-3 → in-context learning, then chat LLMs
- 2022+ — RLHF, instruction tuning, RAG, agents

Top Tips

- start with a pretrained model — don't reinvent the wheel
- tokenizer + model must match!
- look at your data & errors, not just metrics
- be careful with leakage — test set sacred
- document data sources & biases
- for prod: monitor drift, latency, cost

Remember: the field is moving fast. The fundamentals (tokens, embeddings, attention) stay — the specific best model changes every few months. Build intuition, not lock-in

— end of NLP notes —